

13.3. Dynamic Programming

597

13.3.2

DNA and Text Sequence Alignment

A common text-processing problem, which arises in genetics and software engineering, is to test the similarity between two text strings. In a genetics application, the two strings could correspond to two strands of DNA, for which we want to compute similarities. Likewise, in a software engineering application, the two strings could come from two versions of source code for the same program, for which we want to determine changes made from one version to the next. Indeed, determining the similarity between two strings is so common that the Unix and Linux operating systems have a built-in program, named `diff`, for comparing text files.

Given a string $X = x_0x_1x_2 \cdots x_{n-1}$, a subsequence of X is any string that is of the form $x_{i_1}x_{i_2} \cdots x_{i_k}$, where $i_j < i_{j+1}$; that is, it is a sequence of characters that are not necessarily contiguous but are nevertheless taken in order from X . For example, the string `AAAG` is a subsequence of the string `CGATAATTGAGA`.

The DNA and text similarity problem we address here is the longest common subsequence (LCS) problem. In this problem, we are given two character strings, $X = x_0x_1x_2 \cdots x_{n-1}$ and $Y = y_0y_1y_2 \cdots y_{m-1}$, over some alphabet (such as the alphabet $\{A, C, G, T\}$ common in computational genetics) and are asked to find a longest string S that is a subsequence of both X and Y . One way to solve the longest common subsequence problem is to enumerate all subsequences of X and take the largest one that is also a subsequence of Y . Since each character of X is either in or not in a subsequence, there are potentially 2^n different subsequences of X , each of which requires $O(m)$ time to determine whether it is a subsequence of Y . Thus, this brute-force approach yields an exponential-time algorithm that runs in $O(2^n m)$ time, which is very inefficient. Fortunately, the LCS problem is efficiently solvable using dynamic programming.

The Components of a Dynamic Programming Solution

As mentioned above, the dynamic programming technique is used primarily for optimization problems, where we wish to find the best way of doing something.

We can apply the dynamic programming technique in such situations if the problem has certain properties:

Simple Subproblems: There has to be some way of repeatedly breaking the global optimization problem into subproblems. Moreover, there should be a way to parameterize subproblems with just a few indices, like i , j , k , and so on.

Subproblem Optimization: An optimal solution to the global problem must be a composition of optimal subproblem solutions.

Subproblem Overlap: Optimal solutions to unrelated subproblems can contain subproblems in common.

Dynamic Programming

In this section, we discuss the **dynamic programming** algorithm-design technique. This technique is similar to the divide-and-conquer technique (Section 12.2.1), in that it can be applied to a wide variety of different problems. **Dynamic programming** can often be used to take problems that seem to require exponential time and produce polynomial-time algorithms to solve them. In addition, the algorithms that result from applications of the **dynamic programming** technique are usually quite simple—often needing little more than a few lines of code to describe some nested loops for filling in a table.

13.3.1

Matrix Chain-Product

Rather than starting out with an explanation of the general components of the **dynamic programming** technique, we begin by giving a classic, concrete example.

Suppose we are given a collection of n two-dimensional matrices for which we wish to compute the mathematical product

$$A = A_0 \cdot A_1 \cdot A_2 \cdots A_{n-1},$$

where A_i is a $d_i \times d_{i+1}$ matrix, for $i = 0, 1, 2, \dots, n-1$. In the standard matrix multiplication algorithm (which is the one we will use), to multiply a $d \times e$ -matrix B times an $e \times f$ -matrix C , we compute the product, A , as

$$A[i][j] =$$

$$e-1$$

$$\cdot$$

$$k=0$$

$$B[i][k] \cdot C[k][j].$$

This definition implies that matrix multiplication is associative, that is, it implies that $B \cdot (C \cdot D) = (B \cdot C) \cdot D$. Thus, we can parenthesize the expression for A any way we wish and we will end up with the same answer. However, we will not necessarily perform the same number of primitive (that is, scalar) multiplications in each parenthesization, as is illustrated in the following example.

Example 13.4: Let B be a 2×10 -matrix, let C be a 10×50 -matrix, and let D be a 50×20 -matrix. Computing $B \cdot (C \cdot D)$ requires $2 \cdot 10 \cdot 20 + 10 \cdot 50 \cdot 20 = 10400$ multiplications, whereas computing $(B \cdot C) \cdot D$ requires $2 \cdot 10 \cdot 50 + 2 \cdot 50 \cdot 20 = 3000$ multiplications.

The matrix chain-product problem is to determine the parenthesization of the expression defining the product A that minimizes the total number of scalar multiplications performed. As the example above illustrates, the differences between parenthesizations can be dramatic, so finding a good solution can result in significant speedups.

13.3. Dynamic Programming

599

The LCS Algorithm

The definition of $L[j,k]$ satisfies subproblem optimization, for we cannot have a longest common subsequence without also having longest common subsequences for the subproblems. Also, it uses subproblem overlap, because a subproblem solution $L[j,k]$ can be used in several other problems (namely, the problems $L[j+1,k]$, $L[j,k+1]$, and $L[j+1,k+1]$). Turning this definition of $L[j,k]$ into an algorithm is actually quite straightforward. We create an $(n+1) \times (m+1)$ array, L , defined for $0 \leq j \leq n$ and $0 \leq k \leq m$. We initialize all entries to 0, in particular so that all entries of the form $L[j,0]$ and $L[0,k]$ are zero. Then, we iteratively build up values in L until we have $L[n,m]$, the length of a longest common subsequence of X and Y . We give a Python implementation of this algorithm in Code Fragment 13.6.

```
1
def LCS(X, Y):
2
    ...Return table such that L[j][k] is length of LCS for X[0:j] and Y[0:k]....
3
    n, m = len(X), len(Y)
    # introduce convenient notations
4
    L = [[0]
          (m+1) for k in range(n+1)]
    # (n+1) x (m+1) table
5
    for j in range(n):
6
        for k in range(m):
7
            if X[j] == Y[k]:
            # align this match
8
                L[j+1][k+1] = L[j][k] + 1
9
            else:
            # choose to ignore one character
10
                L[j+1][k+1] = max(L[j][k+1], L[j+1][k])
11
    return L
```

Code Fragment 13.6: Dynamic programming algorithm for the LCS problem.

The running time of the algorithm of the LCS algorithm is easy to analyze, for it is dominated by two nested for loops, with the outer one iterating n times and the inner one iterating m times. Since the if-statement and assignment inside the loop each requires $O(1)$ primitive operations, this algorithm runs in $O(nm)$ time. Thus, the dynamic programming technique can be applied to the longest common subsequence problem to improve significantly over the exponential-time brute-force solution to the LCS problem.

The LCS function of Code Fragment 13.6 computes the length of the longest

13.3. Dynamic Programming

595

Defining Subproblems

One way to solve the matrix chain-product problem is to simply enumerate all the possible ways of parenthesizing the expression for A and determine the number of multiplications performed by each one. Unfortunately, the set of all different parenthesizations of the expression for A is equal in number to the set of all different binary trees that have n leaves. This number is exponential in n . Thus, this straightforward (·brute-force·) algorithm runs in exponential time, for there are an exponential number of ways to parenthesize an associative arithmetic expression. We can significantly improve the performance achieved by the brute-force algorithm, however, by making a few observations about the nature of the matrix chain-product problem. The first is that the problem can be split into subproblems. In this case, we can define a number of different subproblems, each of which is to compute the best parenthesization for some subexpression $A_i \cdot A_{i+1} \cdots A_j$. As a concise notation, we use $N_{i,j}$ to denote the minimum number of multiplications needed to compute this subexpression. Thus, the original matrix chain-product problem can be characterized as that of computing the value of $N_{0,n-1}$. This observation is important, but we need one more in order to apply the dynamic programming technique.

Characterizing Optimal Solutions

The other important observation we can make about the matrix chain-product problem is that it is possible to characterize an optimal solution to a particular subproblem in terms of optimal solutions to its subproblems. We call this property the subproblem optimality condition.

In the case of the matrix chain-product problem, we observe that, no matter how we parenthesize a subexpression, there has to be some final matrix multiplication that we perform. That is, a full parenthesization of a subexpression $A_i \cdot A_{i+1} \cdots A_j$ has to be of the form $(A_i \cdots A_k) \cdot (A_{k+1} \cdots A_j)$, for some $k \in \{i, i+1, \dots, j-1\}$. Moreover, for whichever k is the correct one, the products $(A_i \cdots A_k)$ and $(A_{k+1} \cdots A_j)$ must also be solved optimally. If this were not so, then there would be a global optimal that had one of these subproblems solved suboptimally. But this is impossible, since we could then reduce the total number of multiplications by replacing the current subproblem solution by an optimal solution for the subproblem. This observation implies a way of explicitly defining the optimization problem for $N_{i,j}$ in terms of other optimal subproblem solutions. Namely, we can compute $N_{i,j}$ by considering each place k where we could put the final multiplication and taking the minimum over all such choices.

Designing a Dynamic Programming Algorithm

We can therefore characterize the optimal subproblem solution, $N_{i,j}$, as

$$N_{i,j} = \min$$

$$i \leq k < j \{N_{i,k} + N_{k+1,j} + d_{i,k+1}d_{j+1}\},$$

where $N_{i,i} = 0$, since no work is needed for a single matrix. That is, $N_{i,j}$ is the minimum, taken over all possible places to perform the $\cdot$ - n - al multiplication, of the number of multiplications needed to compute each subexpression plus the number of multiplications needed to perform the $\cdot$ - n - al matrix multiplication.

Notice that there is a sharing of subproblems going on that prevents us from dividing the problem into completely independent subproblems (as we would need to do to apply the divide-and-conquer technique). We can, nevertheless, use the equation for $N_{i,j}$ to derive an efficient algorithm by computing $N_{i,j}$ values in a bottom-up fashion, and storing intermediate solutions in a table of $N_{i,j}$ values. We can begin simply enough by assigning $N_{i,i} = 0$ for $i = 0, 1, \dots, n-1$. We can then apply the general equation for $N_{i,j}$ to compute $N_{i,i+1}$ values, since they depend only on $N_{i,i}$ and $N_{i+1,i+1}$ values that are available. Given the $N_{i,i+1}$ values, we can then compute the $N_{i,i+2}$ values, and so on. Therefore, we can build $N_{i,j}$ values up from previously computed values until we can finally compute the value of $N_{0,n-1}$, which is the number that we are searching for. A Python implementation of this dynamic programming solution is given in Code Fragment 13.5; we use techniques from Section 5.6 for representing a multidimensional table in Python.

```

1
def matrix_chain(d):
2
    ...d is a list of n+1 numbers such that size of kth matrix is d[k]-by-d[k+1].
3
4
    Return an n-by-n table such that N[i][j] represents the minimum number of
5
    multiplications needed to compute the product of Ai through Aj inclusive.
6
    ...
7
    n = len(d) - 1
    # number of matrices
8
    N = [[0]
    n for i in range(n)]
    # initialize n-by-n result to zero
9
    for b in range(1, n):
    # number of products in subchain
10
        for i in range(n-b):
        # start of subchain
11
            j = i + b

```

740

Index

Cormen, Thomas, 535, 696

Counter class, 450

CPU, 111

CRC cards, 63

CreditCard class, 63, 69·73, 73, 83·86

Crochemore, Maxime, 618

cryptography, 216·218

ctypes module, 191, 195

cubic function, 119

cyber-dollar, 197·199, 497·500, 682

cycle, 623

directed, 623

cyclic-shift hash code, 413·414

DAG, see directed acyclic graph

data packets, 227

data structure, 110

Dawson, Michael, 55

debugging, 62

decision tree, 311, 463, 562

decorate-sort-undecorate design pattern,
570

decrease-and-conquer, 571

decryption, 216

deep copy, 102, 188

deepcopy function, 102, 188

def keyword, 23

degree of a vertex, 621

del operator, 15, 75

DeMorgan's Law, 137

Demurjian, Steven, 108, 254

depth of a tree, 308·310

depth-first search (DFS), 639·647

deque, 247·249

abstract data type, 247·248

linked-list implementation, 249, 275

deque class, 249, 251

descendant, 302

design patterns, v, 61

adapter, 231

amortization, 197·200

brute force, 584

composition, 287, 365, 407

divide-and-conquer, 538·542, 550·
551

dynamic programming, 594·600

factory method, 479

greedy method, 603

2.1. Goals, Principles, and Patterns

61

2.1.3

Design Patterns

Object-oriented design facilitates reusable, robust, and adaptable software. Designing good code takes more than simply understanding object-oriented methodologies, however. It requires the effective use of object-oriented design techniques. Computing researchers and practitioners have developed a variety of organizational concepts and methodologies for designing quality object-oriented software that is concise, correct, and reusable. Of special relevance to this book is the concept of a design pattern, which describes a solution to a typical software design problem. A pattern provides a general template for a solution that can be applied in many different situations. It describes the main elements of a solution in an abstract way that can be specialized for a specific problem at hand. It consists of a name, which identifies the pattern; a context, which describes the scenarios for which this pattern can be applied; a template, which describes how the pattern is applied; and a result, which describes and analyzes what the pattern produces.

We present several design patterns in this book, and we show how they can be consistently applied to implementations of data structures and algorithms. These design patterns fall into two groups—patterns for solving algorithm design problems and patterns for solving software engineering problems. The algorithm design patterns we discuss include the following:

- Recursion (Chapter 4)
- Amortization (Sections 5.3 and 11.4)
- Divide-and-conquer (Section 12.2.1)
- Prune-and-search, also known as decrease-and-conquer (Section 12.7.1)
- Brute force (Section 13.2.1)
- **Dynamic programming** (Section 13.3).
- The greedy method (Sections 13.4.2, 14.6.2, and 14.7)

Likewise, the software engineering design patterns we discuss include:

- Iterator (Sections 1.8 and 2.3.4)
- Adapter (Section 6.1.2)
- Position (Sections 7.4 and 8.1.2)
- Composition (Sections 7.6.1, 9.2.1, and 10.1.4)
- Template method (Sections 2.4.3, 8.4.6, 10.1.3, 10.5.2, and 11.2.1)
- Locator (Section 9.5.1)
- Factory method (Section 11.2.1)

Rather than explain each of these concepts here, however, we introduce them throughout the text as noted above. For each pattern, be it for algorithm engineering or software engineering, we explain its general use and we illustrate it with at least one concrete example.

Chapter

13

Text Processing

Contents

13.1 Abundance of Digitized Text	
582	
13.1.1 Notations for Strings and the Python str Class	583
13.2 Pattern-Matching Algorithms	
584	
13.2.1 Brute Force	584
13.2.2 The Boyer-Moore Algorithm	
.....	586
13.2.3 The Knuth-Morris-Pratt Algorithm	590
13.3 Dynamic Programming	
.....	
594	
13.3.1 Matrix Chain-Product	594
13.3.2 DNA and Text Sequence Alignment	
.....	597
13.4 Text Compression and the Greedy Method	
601	
13.4.1 The Hu·man Coding Algorithm	
.....	602
13.4.2 The Greedy Method	603
13.5 Tries	
604	
13.5.1 Standard Tries	604
13.5.2 Compressed Tries	608
13.5.3 Su·x Tries	
.....	610
13.5.4 Search Engine Indexing	612
13.6 Exercises	
.....	
613	

Abundance of Digitized Text

Despite the wealth of multimedia information, text processing remains one of the dominant functions of computers. Computer are used to edit, store, and display documents, and to transport documents over the Internet. Furthermore, digital systems are used to archive a wide range of textual information, and new data is being generated at a rapidly increasing pace. A large corpus can readily surpass a petabyte of data (which is equivalent to a thousand terabytes, or a million gigabytes). Common examples of digital collections that include textual information are:

- Snapshots of the World Wide Web, as Internet document formats HTML and XML are primarily text formats, with added tags for multimedia content
- All documents stored locally on a user's computer
- Email archives
- Customer reviews
- Compilations of status updates on social networking sites such as Facebook
- Feeds from microblogging sites such as Twitter and Tumblr

These collections include written text from hundreds of international languages. Furthermore, there are large data sets (such as DNA) that can be viewed computationally as ·strings· even though they are not language.

In this chapter we explore some of the fundamental algorithms that can be used to efficiently analyze and process large textual data sets. In addition to having interesting applications, text-processing algorithms also highlight some important algorithmic design patterns.

We begin by examining the problem of searching for a pattern as a substring of a larger piece of text, for example, when searching for a word in a document. The pattern-matching problem gives rise to the brute-force method, which is often inefficient but has wide applicability.

Next, we introduce an algorithmic technique known as **dynamic programming**, which can be applied in certain settings to solve a problem in polynomial time that appears at first to require exponential time to solve. We demonstrate the application on this technique to the problem of finding partial matches between strings that may be similar but not perfectly aligned. This problem arises when making suggestions for a misspelled word, or when trying to match related genetic samples.

Because of the massive size of textual data sets, the issue of compression is important, both in minimizing the number of bits that need to be communicated through a network and to reduce the long-term storage requirements for archives. For text compression, we can apply the greedy method, which often allows us to approximate solutions to hard problems, and for some problems (such as in text compression) actually gives rise to optimal algorithms.

Finally, we examine several special-purpose data structures that can be used to better organize textual data in order to support more efficient run-time queries.

Applying Dynamic Programming to the LCS Problem

Recall that in the LCS problem, we are given two character strings, X and Y , of length n and m , respectively, and are asked to find a longest string S that is a subsequence of both X and Y . Since X and Y are character strings, we have a natural set of indices with which to define subproblems: indices into the strings X and Y . Let us define a subproblem, therefore, as that of computing the value $L_{j,k}$, which we will use to denote the length of a longest string that is a subsequence of both prefixes $X[0:j]$ and $Y[0:k]$. This definition allows us to rewrite $L_{j,k}$ in terms of optimal subproblem solutions. This definition depends on which of two cases we are in. (See Figure 13.7.)

$$L_{10,12} = 1 + L_{9,11}$$

$$L_{9,11} = \max(L_{9,10}, L_{8,11})$$

6 7 8 9 10 11

G T T C C T A A T A

C

A T A A T T G

G

A

G

A

0 1 2 3 4 5 6 7 8 9

X =

0

Y =

1 2 3 4 5

5 6 7 8 9 10

G T T C C T A A T

C

A T A A T T G

G

G

A

0 1 2 3 4 5 6 7 8

X =

0

Y =

1 2 3 4

(a)

(b)

Figure 13.7: The two cases in the longest common subsequence algorithm for computing $L_{j,k}$: (a) $x_{j+1} = y_{k+1}$; (b) $x_{j+1} \neq y_{k+1}$.

• $x_{j+1} = y_{k+1}$. In this case, we have a match between the last character of $X[0:j]$ and the last character of $Y[0:k]$. We claim that this character belongs to a longest common subsequence of $X[0:j]$ and $Y[0:k]$. To justify this claim, let us suppose it is not true. There has to be some longest common subsequence $x_1 x_2 \dots x_a c = y_1 y_2 \dots y_b c$. If $x_a c = x_{j+1}$ or $y_b c = y_{k+1}$, then we get the same sequence by setting $a = j+1$ and $b = k+1$. Alter-